

Verifiche indipendenti (spettro e stati) — trimero a catena aperta

Note di lavoro — Tesi triennale, Samuele Schiavi
Relatore: Prof. Alessandro Chiesa

Introduzione

Questo documento raccoglie i due script di verifica indipendente per il trimero a catena aperta: `verifica_catena.py` (spettro, campo critico, simmetrie — prima chiamato `verifica_catena_preliminare.py`, stesso principio di `verifica_anello.py` per l'anello) e `verifica_stati_catena.py` (otto stati di Kambe espliciti, verificati come vettori — stesso principio di `verifica_stati_anello.py`). Entrambi sono script standalone, indipendenti dal modulo `trimer_chain_exact.py`: non lo importano, ricostruiscono l'Hamiltoniana e gli stati da zero con la propria implementazione.

Come già osservato per la coppia gemella dell'anello, i due script hanno scopi diversi nonostante il nome simile: il primo lavora solo con lo spettro, il secondo solo con gli stati espliciti.

`verifica_catena.py`

Script di controllo indipendente dal modulo principale (fino a poco fa chiamato `verifica_catena_preliminare.py`): spettro numerico su 200 configurazioni casuali di (J, b) contro la forma chiusa, controllo della traccia (deve essere nulla), verifica della simmetria P_{13} (deve commutare con H nel caso uniforme $J_{12} = J_{23}$, e **non** commutare se $J_{12} \neq J_{23}$ — controllato esplicitamente in entrambi i casi), campo critico $b_c = 3J$ trovato numericamente via il punto di massima curvatura dello spettro, e verifica che per $J < 0$ non ci sia alcun incrocio (curvatura sempre nulla).

Funzioni interne.

- `op(site, P)` — colloca l'operatore di Pauli P sul sito indicato (1, 2 o 3) dei 3 qubit, identità sugli altri due.
- `dot(i, j)` — il prodotto scalare di spin $\sigma_i \cdot \sigma_j = X_i X_j + Y_i Y_j + Z_i Z_j$.
- `H_chain(J, b)` — l'Hamiltoniana della catena uniforme, $J(\sigma_1 \cdot \sigma_2 + \sigma_2 \cdot \sigma_3) + b \sum_i Z_i$ (un solo parametro J , nessun J').
- `perm_13()` — la matrice di permutazione 8×8 che scambia i qubit 1 e 3, costruita esplicitamente bit a bit (non tramite libreria).
- `H_chain_nonuniform(J12, J23, b)` — variante della catena con le due costanti di accoppiamento indipendenti, usata solo per il test di rottura della simmetria (§ punto 4 sotto).

Struttura dello script (6 blocchi di test, eseguiti in sequenza, non incapsulati in funzioni separate).

1. Spettro numerico vs formula chiusa su 200 punti (J, b) casuali.
2. Traccia di H (deve annullarsi: somma di matrici di Pauli).

3. Commutatore $[P_{13}, H]$ nel caso uniforme ($J_{12} = J_{23}$): deve essere esattamente zero.
4. Commutatore $[P_{13}, H]$ nel caso non uniforme ($J_{12} \neq J_{23}$): deve essere **diverso** da zero — verifica che la simmetria si rompa davvero quando ci si aspetta che si rompa, non solo che valga quando dovrebbe valere.
5. Campo critico numerico (ricerca del punto di massima curvatura, cioè del "kink", nello spettro del fondamentale in funzione di b) confrontato con la formula $b_c = 3J$.
6. Caso $J < 0$: verifica che la curvatura resti nulla ovunque (nessun incrocio), coerente con la teoria.

Come si usa. Nessun argomento da riga di comando, nessun import locale: `python3 verifica_catena.py`. Stampa a schermo i risultati numerici dei 6 blocchi (non ci sono etichette OK/FALLITO esplicite come nello script dell'anello — i valori vanno confrontati a occhio con gli attesi, riportati nei commenti del codice).

Verifiche. Rieseguito in questa sessione: errore massimo sullo spettro 5.3×10^{-15} , traccia $8.9 \times 10^{-16} \approx 0$, $\| [P_{13}, H] \| = 0.0$ esatto nel caso uniforme, $\| [P_{13}, H] \| = 0.8$ (chiaramente non nullo) nel caso non uniforme, campo critico numerico = 3.0 esatto contro $3J = 3.0$ atteso, curvatura massima per $J < 0$ pari a $7.1 \times 10^{-15} \approx 0$.

Conclusione. Stesso ruolo di `verifica_anello.py` per l'anello: controllo complementare ai self-test interni di `trimer_chain_exact.py`, con un accento in più specifico della catena — la verifica esplicita che la simmetria P_{13} si rompa nel caso non uniforme, non solo che valga in quello uniforme.

verifica_stati_catena.py

Script di controllo indipendente che costruisce esplicitamente gli **otto stati di Kambe** in base computazionale e li verifica uno per uno: norma, energia (numerica contro teorica), residuo come autovettore esatto, e ortogonalità reciproca (matrice di Gram).

Funzioni interne.

- `op(site, P)` — stessa funzione di `verifica_catena.py`.
- `dot(i, j)` — stessa funzione di `verifica_catena.py`.
- `H_chain(J, b)` — stessa Hamiltoniana di `verifica_catena.py`, ridefinita indipendentemente in questo file.
- `ket(bits)` — lo stato della base computazionale corrispondente a una stringa di 3 bit, come vettore $\mathbb{C}^8$.
- `state(coeffs)` — costruisce una sovrapposizione pesata a partire da una lista di coppie (ampiezza, stringa di bit); restituisce il vettore e la sua norma.
- Il modulo non definisce una funzione dedicata per l'energia teorica (a differenza di `verifica_stati_anello.py`): la calcola inline, dentro il ciclo principale, con la stessa formula $E(S_{13}, S, M) = 2J[S(S+1) - S_{13}(S_{13}+1) - \frac{3}{4}] + 2bM$.

Dati interni (non funzioni).

- `states` — dizionario che mappa (blocco, M) alla lista di coefficienti espliciti dell'autostato corrispondente, trascritti da `derivazione_stati_kambe_trimero_catena.pdf`.
- `QN` — dizionario che mappa ogni blocco alla coppia (S_{13}, S) .

Come si usa. Nessun argomento, nessun import locale: `python3 verifica_stati_catena.py`. Punto di lavoro fissato nello script ($J = 1.3$, $b = -0.7$). Stampa, per ciascuno degli otto stati: norma al quadrato, energia numerica e teorica affiancate, poi il residuo $\|Hv - Ev\|$; infine il massimo elemento fuori diagonale e il massimo scarto dalla diagonale della matrice di Gram.

Verifiche. Eseguito in questa sessione: tutti gli otto stati con norma $^2 = 1.000000000$, energia numerica identica a quella teorica, residuo fra 0 e 9.9×10^{-16} , matrice di Gram diagonale a 6.7×10^{-17} .

Conclusione. A differenza dell'anello (dove questa verifica era assente come script standalone, solo incorporata nei self-test del modulo principale), per la catena questo script esisteva già prima di questa sessione — è servito da modello diretto per costruire `verifica_stati_anello.py`.

Conclusione generale

I due script coprono aspetti complementari e non sovrapposti: spettro/campo critico/simmetrie da un lato, stati espliciti come vettori dall'altro — stessa struttura logica della coppia gemella per l'anello (documento gemello), con l'aggiunta specifica della catena del test esplicito di rottura della simmetria P_{13} nel caso non uniforme.